Perceptron, Logistic Regression & Softmax

Linear Classifiers: The Math

Timothy Obiso

NLP II Recitation
Brandeis University

January 30, 2026

Today's Goals

You've done the readings (J&M Ch. 3–4, Eisenstein Ch. 2).

Today we'll:

- Derive the perceptron update rule from first principles
- Derive cross-entropy loss from maximum likelihood
- Compute gradients for logistic regression by hand
- Extend to multi-class with softmax
- Connect everything: why does $(p - y) \cdot \mathbf{x}$ keep showing up?

Perceptron: The Setup

Binary classification: Given input $\mathbf{x} \in \mathbb{R}^d$, predict $y \in \{-1, +1\}$

Perceptron: The Setup

Binary classification: Given input $\mathbf{x} \in \mathbb{R}^d$, predict $y \in \{-1, +1\}$

Model:

$$\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = \text{sign}\left(\sum_{j=1}^{d} w_j x_j + b\right)$$

Perceptron: The Setup

Binary classification: Given input $\mathbf{x} \in \mathbb{R}^d$, predict $y \in \{-1, +1\}$

Model:

$$\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = \text{sign}\left(\sum_{j=1}^{d} w_j x_j + b\right)$$

Notational trick: Absorb bias into weights

- Append 1 to input: $\mathbf{x} \leftarrow [x_1, \ldots, x_d, 1]$
- Append b to weights: $\mathbf{w} \leftarrow [w_1, \ldots, w_d, b]$
- Now: $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x})$

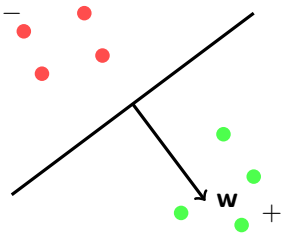
Geometric Intuition

The weight vector $\mathbf{w}$ defines a **hyperplane**:

$$\{\mathbf{x} : \mathbf{w} \cdot \mathbf{x} = 0\}$$

Key insight: $\mathbf{w}$ is **normal** to this hyperplane

Points on the $+$ side: $\mathbf{w} \cdot \mathbf{x} > 0$
Points on the $-$ side: $\mathbf{w} \cdot \mathbf{x} < 0$



The Perceptron Update Rule

When do we update?

The Perceptron Update Rule

When do we update? Only on mistakes.

A mistake occurs when $y \neq \hat{y}$, equivalently when:

$$y(\mathbf{w} \cdot \mathbf{x}) \leq 0$$

The Perceptron Update Rule

When do we update? Only on mistakes.

A mistake occurs when $y \neq \hat{y}$, equivalently when:

$$y(\mathbf{w} \cdot \mathbf{x}) \leq 0$$

The update:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y \cdot \mathbf{x}$$

The Perceptron Update Rule

When do we update? Only on mistakes.

A mistake occurs when $y \neq \hat{y}$, equivalently when:

$$y(\mathbf{w} \cdot \mathbf{x}) \leq 0$$

The update:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y \cdot \mathbf{x}$$

Why does this make sense?

- If $y = +1$ but we predicted $-$: add $\mathbf{x}$ to $\mathbf{w}$ (rotate toward $\mathbf{x}$)
- If $y = -1$ but we predicted $+$: subtract $\mathbf{x}$ from $\mathbf{w}$ (rotate away from $\mathbf{x}$)

Why the Update Works: Geometric View

After update, the new prediction on $\mathbf{x}$ is:

$$\begin{aligned}\mathbf{w}^{(t+1)} \cdot \mathbf{x} &= (\mathbf{w}^{(t)} + y\mathbf{x}) \cdot \mathbf{x} \\ &= \mathbf{w}^{(t)} \cdot \mathbf{x} + y(\mathbf{x} \cdot \mathbf{x}) \\ &= \mathbf{w}^{(t)} \cdot \mathbf{x} + y\|\mathbf{x}\|^2\end{aligned}$$

Since $\|\mathbf{x}\|^2 > 0$:

- If $y = +1$: new score is **higher** (more positive)
- If $y = -1$: new score is **lower** (more negative)

We move in the right direction, but not necessarily enough to fix this example!

The Perceptron Algorithm

Initialize $\mathbf{w} = \mathbf{0}$
repeat
  for each $(\mathbf{x}_i, y_i)$ in training data **do**
    $\hat{y}_i \leftarrow \text{sign}(\mathbf{w} \cdot \mathbf{x}_i)$
    if $y_i \neq \hat{y}_i$ **then** ▷ Misclassified
      $\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$
    end if
  end for
until no mistakes in a full pass (or max iterations)
return w

Equivalent condition: For $y \in \{-1, +1\}$, "$y_i \neq \hat{y}_i$" is equivalent to:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i) \leq 0$$

Why? If $y_i = +1$ but $\hat{y}_i = -1$, then $\mathbf{w} \cdot \mathbf{x}_i < 0$, so $y_i(\mathbf{w} \cdot \mathbf{x}_i) < 0$. Same logic for the other case.

Problem: Perceptron Instability

Issue: Final weights depend heavily on the last few updates

Problem: Perceptron Instability

Issue: Final weights depend heavily on the last few updates

Example: Training for 10 epochs on 1000 examples

- Epoch 10, example 997 causes an update
- This single update has outsized influence on $\mathbf{w}_{\text{final}}$
- Earlier "good" weight vectors are forgotten

Problem: Perceptron Instability

Issue: Final weights depend heavily on the last few updates

Example: Training for 10 epochs on 1000 examples

- Epoch 10, example 997 causes an update
- This single update has outsized influence on $\mathbf{w}_{\text{final}}$
- Earlier "good" weight vectors are forgotten

Consequence:

- Different random shuffles $\Rightarrow$ very different final weights
- Weights oscillate near the decision boundary
- Overfits to the last few examples seen

Averaged Perceptron

Idea: Return the **average** of all weight vectors seen during training

Averaged Perceptron

Idea: Return the **average** of all weight vectors seen during training

Implementation: Maintain a running sum

Initialize $\mathbf{w} = \mathbf{0}$, $\mathbf{w}_{\text{sum}} = \mathbf{0}$, $t = 0$
for each epoch **do**
    for each $(\mathbf{x}_i, y_i)$ in training data **do**
        $t \leftarrow t + 1$
        if $y_i \neq \text{sign}(\mathbf{w} \cdot \mathbf{x}_i)$ **then**
            $\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$
        end if
        $\mathbf{w}_{\text{sum}} \leftarrow \mathbf{w}_{\text{sum}} + \mathbf{w}$ ▷ Every example!
    end for
end for
return $\mathbf{w}_{\text{sum}}/t$

Why Averaging Works

Intuition: Weights that "survive" many examples get more votes

Why Averaging Works

Intuition: Weights that "survive" many examples get more votes

Example:

- Good $\mathbf{w}$ found at step 500, survives until step 900 (400 steps)
- Bad update at step 900, new $\mathbf{w}'$ only survives 100 more steps
- Average weights $\mathbf{w}$ $4\times$ more than $\mathbf{w}'$

Why Averaging Works

Intuition: Weights that "survive" many examples get more votes

Example:

- Good $\mathbf{w}$ found at step 500, survives until step 900 (400 steps)
- Bad update at step 900, new $\mathbf{w}'$ only survives 100 more steps
- Average weights $\mathbf{w}$ $4\times$ more than $\mathbf{w}'$

In practice:

- More stable across different random shuffles
- Often 1–2% accuracy improvement
- No extra hyperparameters
- Almost free — just one extra vector addition per example

Perceptron Limitations

What if data isn't linearly separable?

- Perceptron will never converge
- Cycles through the data forever

No probabilistic interpretation:

- Output is just ± 1, no confidence
- How sure are we about this prediction?

No real loss function:

- Hard to extend or regularize
- Can't do gradient-based optimization

Enter: Logistic Regression

From Hard to Soft Decisions

Perceptron: $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x})$ (hard ± 1)

From Hard to Soft Decisions

Perceptron: $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x})$ (hard ± 1)

Logistic Regression: Output a **probability**

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x})$$

From Hard to Soft Decisions

Perceptron: $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x})$ (hard ± 1)

Logistic Regression: Output a **probability**

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x})$$

where σ is the **sigmoid** (logistic) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

From Hard to Soft Decisions

Perceptron: $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x})$ (hard ± 1)

Logistic Regression: Output a **probability**

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x})$$

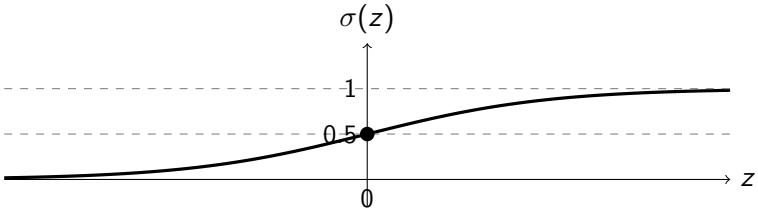
where σ is the **sigmoid** (logistic) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

Properties of sigmoid:

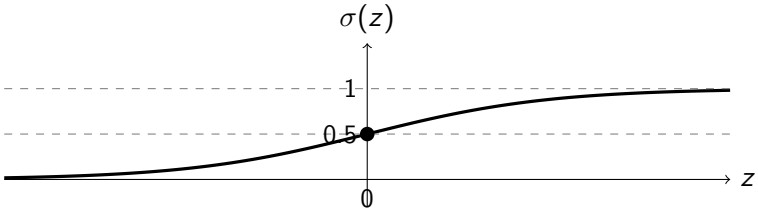
- Maps $\mathbb{R} \to (0, 1)$
- $\sigma(0) = 0.5$, $\sigma(\infty) = 1$, $\sigma(-\infty) = 0$
- Symmetric: $\sigma(-z) = 1 - \sigma(z)$

The Sigmoid Function



Crucial derivative:

The Sigmoid Function



Crucial derivative: $\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$

(We'll use this when computing gradients!)

Why Sigmoid? The Log-Odds View

Define: Let $p = P(y = 1 \mid \mathbf{x})$

Why Sigmoid? The Log-Odds View

Define: Let $p = P(y = 1 \mid \mathbf{x})$

Log-odds (logit):

$$\log \frac{p}{1-p} = \mathbf{w} \cdot \mathbf{x}$$

This is a **linear** function of $\mathbf{x}$!

Why Sigmoid? The Log-Odds View

Define: Let $p = P(y = 1 \mid \mathbf{x})$

Log-odds (logit):

$$\log \frac{p}{1-p} = \mathbf{w} \cdot \mathbf{x}$$

This is a **linear** function of $\mathbf{x}$!

Solving for p:

$$\begin{aligned} \frac{p}{1-p} &= e^{\mathbf{w}\cdot\mathbf{x}} \\ p &= (1-p)e^{\mathbf{w}\cdot\mathbf{x}} \\ p(1+e^{\mathbf{w}\cdot\mathbf{x}}) &= e^{\mathbf{w}\cdot\mathbf{x}} \\ p &= \frac{e^{\mathbf{w}\cdot\mathbf{x}}}{1+e^{\mathbf{w}\cdot\mathbf{x}}} = \sigma(\mathbf{w}\cdot\mathbf{x}) \end{aligned}$$

Deriving the Loss: Maximum Likelihood

Setup: Labels $y \in \{0, 1\}$, model gives $p_i = \sigma(\mathbf{w} \cdot \mathbf{x}_i)$

Deriving the Loss: Maximum Likelihood

Setup: Labels $y \in \{0, 1\}$, model gives $p_i = \sigma(\mathbf{w} \cdot \mathbf{x}_i)$

Likelihood of one example:

$$P(y_i \mid \mathbf{x}_i; \mathbf{w}) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

Deriving the Loss: Maximum Likelihood

Setup: Labels $y \in \{0, 1\}$, model gives $p_i = \sigma(\mathbf{w} \cdot \mathbf{x}_i)$

Likelihood of one example:

$$P(y_i \mid \mathbf{x}_i; \mathbf{w}) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

(Check: if $y_i = 1$, this is p_i; if $y_i = 0$, this is $1 - p_i$)

Deriving the Loss: Maximum Likelihood

Setup: Labels $y \in \{0, 1\}$, model gives $p_i = \sigma(\mathbf{w} \cdot \mathbf{x}_i)$

Likelihood of one example:

$$P(y_i \mid \mathbf{x}_i; \mathbf{w}) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

(Check: if $y_i = 1$, this is p_i; if $y_i = 0$, this is $1 - p_i$)

Likelihood of all data (assuming i.i.d.):

$$\mathcal{L}(\mathbf{w}) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}$$

From Likelihood to Cross-Entropy Loss

Log-likelihood (easier to work with):

$$\ell(\mathbf{w}) = \log \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

From Likelihood to Cross-Entropy Loss

Log-likelihood (easier to work with):

$$\ell(\mathbf{w}) = \log \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Maximum likelihood = minimize **negative log-likelihood**:

$$\boxed{\mathcal{J}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]}$$

From Likelihood to Cross-Entropy Loss

Log-likelihood (easier to work with):

$$\ell(\mathbf{w}) = \log \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Maximum likelihood = minimize **negative log-likelihood**:

$$\boxed{\mathcal{J}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]}$$

This is the **binary cross-entropy loss**!

Also called "log loss" — same thing, different names.

Understanding Cross-Entropy

For a single example with true label y and predicted probability p:

$$\mathcal{J} = -[y \log p + (1 - y) \log(1 - p)]$$

Understanding Cross-Entropy

For a single example with true label y and predicted probability p:

$$\mathcal{J} = -[y \log p + (1 - y) \log(1 - p)]$$

Case 1: $y = 1$ (true positive)

$$\mathcal{J} = -\log p$$

Loss is low when $p \to 1$, high when $p \to 0$

Understanding Cross-Entropy

For a single example with true label y and predicted probability p:

$$\mathcal{J} = -[y \log p + (1 - y) \log(1 - p)]$$

Case 1: $y = 1$ (true positive)

$$\mathcal{J} = -\log p$$

Loss is low when $p \to 1$, high when $p \to 0$

Case 2: $y = 0$ (true negative)

$$\mathcal{J} = -\log(1 - p)$$

Loss is low when $p \to 0$, high when $p \to 1$

Understanding Cross-Entropy

For a single example with true label y and predicted probability p:

$$\mathcal{J} = -[y \log p + (1 - y) \log(1 - p)]$$

Case 1: $y = 1$ (true positive)

$$\mathcal{J} = -\log p$$

Loss is low when $p \to 1$, high when $p \to 0$

Case 2: $y = 0$ (true negative)

$$\mathcal{J} = -\log(1 - p)$$

Loss is low when $p \to 0$, high when $p \to 1$

Confident wrong predictions are heavily penalized!

Computing the Gradient

Let's derive $\frac{\partial \mathcal{J}}{\partial w_j}$ for a single example.

Computing the Gradient

Let's derive $\frac{\partial \mathcal{J}}{\partial w_j}$ for a single example.

Let $z = \mathbf{w} \cdot \mathbf{x}$ and $p = \sigma(z)$. Using chain rule:

Computing the Gradient

Let's derive $\frac{\partial \mathcal{J}}{\partial w_j}$ for a single example.

Let $z = \mathbf{w} \cdot \mathbf{x}$ and $p = \sigma(z)$. Using chain rule:

$$\frac{\partial \mathcal{J}}{\partial w_j} = \frac{\partial \mathcal{J}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

Computing the Gradient

Let's derive $\frac{\partial \mathcal{J}}{\partial w_j}$ for a single example.

Let $z = \mathbf{w} \cdot \mathbf{x}$ and $p = \sigma(z)$. Using chain rule:

$$\frac{\partial \mathcal{J}}{\partial w_j} = \frac{\partial \mathcal{J}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

Step 1: $\frac{\partial z}{\partial w_j} = x_j$

Computing the Gradient

Let's derive $\frac{\partial \mathcal{J}}{\partial w_j}$ for a single example.

Let $z = \mathbf{w} \cdot \mathbf{x}$ and $p = \sigma(z)$. Using chain rule:

$$\frac{\partial \mathcal{J}}{\partial w_j} = \frac{\partial \mathcal{J}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

Step 1: $\frac{\partial z}{\partial w_j} = x_j$

Step 2: $\frac{\partial p}{\partial z} = \sigma(z)(1 - \sigma(z)) = p(1 - p)$

Computing the Gradient

Let's derive $\frac{\partial \mathcal{J}}{\partial w_j}$ for a single example.

Let $z = \mathbf{w} \cdot \mathbf{x}$ and $p = \sigma(z)$. Using chain rule:

$$\frac{\partial \mathcal{J}}{\partial w_j} = \frac{\partial \mathcal{J}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

Step 1: $\frac{\partial z}{\partial w_j} = x_j$

Step 2: $\frac{\partial p}{\partial z} = \sigma(z)(1 - \sigma(z)) = p(1 - p)$

Step 3: $\frac{\partial \mathcal{J}}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{p-y}{p(1-p)}$

The Beautiful Gradient

Putting it together:

$$\begin{aligned}\frac{\partial \mathcal{J}}{\partial w_j} &= \frac{p-y}{p(1-p)} \cdot p(1-p) \cdot x_j \\ &= (p-y) \cdot x_j \\ &= (\sigma(\mathbf{w} \cdot \mathbf{x}) - y) \cdot x_j\end{aligned}$$

In vector form:

$$\boxed{\nabla_{\mathbf{w}} \mathcal{J} = (\sigma(\mathbf{w} \cdot \mathbf{x}) - y) \cdot \mathbf{x} = (p-y) \cdot \mathbf{x}}$$

Remarkably simple! Gradient $\propto$ (prediction $-$ truth) $\times$ input

Gradient Descent Update

Batch gradient descent:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)\mathbf{x}_i$$

Gradient Descent Update

Batch gradient descent:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)\mathbf{x}_i$$

Stochastic gradient descent (SGD):

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot (p_i - y_i)\mathbf{x}_i$$

where η is the **learning rate**

Gradient Descent Update

Batch gradient descent:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)\mathbf{x}_i$$

Stochastic gradient descent (SGD):

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot (p_i - y_i)\mathbf{x}_i$$

where η is the **learning rate**

Compare to perceptron: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i$ (only on mistakes)

Gradient Descent Update

Batch gradient descent:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)\mathbf{x}_i$$

Stochastic gradient descent (SGD):

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot (p_i - y_i)\mathbf{x}_i$$

where η is the **learning rate**

Compare to perceptron: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i$ (only on mistakes)

Logistic regression: smooth updates weighted by confidence; Perceptron: binary updates only on errors

Regularization

Problem: If data is linearly separable, $\|\mathbf{w}\| \to \infty$

(Pushing sigmoid outputs toward 0 and 1)

Regularization

Problem: If data is linearly separable, $\|\mathbf{w}\| \to \infty$

(Pushing sigmoid outputs toward 0 and 1)

Solution: L2 regularization

$$\mathcal{J}_{\text{reg}}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

Regularization

Problem: If data is linearly separable, $\|\mathbf{w}\| \to \infty$

(Pushing sigmoid outputs toward 0 and 1)

Solution: L2 regularization

$$\mathcal{J}_{\text{reg}}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

New gradient:

$$\nabla \mathcal{J}_{\text{reg}} = \nabla \mathcal{J} + \lambda \mathbf{w}$$

Regularization

Problem: If data is linearly separable, $\|\mathbf{w}\| \to \infty$

(Pushing sigmoid outputs toward 0 and 1)

Solution: L2 regularization

$$\mathcal{J}_{\text{reg}}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

New gradient:

$$\nabla \mathcal{J}_{\text{reg}} = \nabla \mathcal{J} + \lambda \mathbf{w}$$

Effect: Penalizes large weights, encourages simpler models

λ controls trade-off between fit and simplicity

From Binary to Multi-class

Logistic regression: 2 classes, output $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x})$

From Binary to Multi-class

Logistic regression: 2 classes, output $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x})$

What if we have K classes?

From Binary to Multi-class

Logistic regression: 2 classes, output $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x})$

What if we have K classes?

Setup:

- One weight vector per class: $\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_K$
- Raw scores: $z_k = \mathbf{w}_k \cdot \mathbf{x}$ for each class k
- Need to convert scores to probabilities that sum to 1

From Binary to Multi-class

Logistic regression: 2 classes, output $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x})$

What if we have K classes?

Setup:

- One weight vector per class: $\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_K$
- Raw scores: $z_k = \mathbf{w}_k \cdot \mathbf{x}$ for each class k
- Need to convert scores to probabilities that sum to 1

Enter: the softmax function

The Softmax Function

Definition:

$$P(y = k \mid \mathbf{x}) = \text{softmax}(z_1, \ldots, z_K)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

The Softmax Function

Definition:

$$P(y = k \mid \mathbf{x}) = \text{softmax}(z_1, \ldots, z_K)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Properties:

- All outputs in $(0, 1)$
- Outputs sum to 1: $\sum_{k=1}^{K} P(y = k \mid \mathbf{x}) = 1$
- Preserves ordering: larger $z_k \Rightarrow$ larger $P(y = k)$
- "Soft" version of argmax

The Softmax Function

Definition:

$$P(y = k \mid \mathbf{x}) = \text{softmax}(z_1, \ldots, z_K)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Properties:

- All outputs in $(0, 1)$
- Outputs sum to 1: $\sum_{k=1}^{K} P(y = k \mid \mathbf{x}) = 1$
- Preserves ordering: larger $z_k \Rightarrow$ larger $P(y = k)$
- "Soft" version of argmax

Connection to sigmoid: For $K = 2$, softmax reduces to sigmoid! (Try it)

Multi-class Cross-Entropy Loss

One-hot encoding: True label y becomes vector $\mathbf{y}$ where

$$y_k = \begin{cases} 1 & \text{if } k = \text{true class} \\ 0 & \text{otherwise} \end{cases}$$

Multi-class Cross-Entropy Loss

One-hot encoding: True label y becomes vector $\mathbf{y}$ where

$$y_k = \begin{cases} 1 & \text{if } k = \text{true class} \\ 0 & \text{otherwise} \end{cases}$$

Cross-entropy loss:

$$\mathcal{J} = -\sum_{k=1}^{K} y_k \log p_k$$

Multi-class Cross-Entropy Loss

One-hot encoding: True label y becomes vector $\mathbf{y}$ where

$$y_k = \begin{cases} 1 & \text{if } k = \text{true class} \\ 0 & \text{otherwise} \end{cases}$$

Cross-entropy loss:

$$\mathcal{J} = -\sum_{k=1}^{K} y_k \log p_k$$

Since only one $y_k = 1$, this simplifies to:

$$\mathcal{J} = -\log p_c$$

where c is the true class.

Multi-class Cross-Entropy Loss

One-hot encoding: True label y becomes vector $\mathbf{y}$ where

$$y_k = \begin{cases} 1 & \text{if } k = \text{true class} \\ 0 & \text{otherwise} \end{cases}$$

Cross-entropy loss:

$$\mathcal{J} = -\sum_{k=1}^{K} y_k \log p_k$$

Since only one $y_k = 1$, this simplifies to:

$$\mathcal{J} = -\log p_c$$

where c is the true class.

(Same intuition: penalize low probability on correct class)

Softmax Gradient: The Punchline

Claim: For the correct class c:

$$\frac{\partial \mathcal{J}}{\partial z_k} = p_k - y_k = \begin{cases} p_k - 1 & \text{if } k = c \\ p_k & \text{if } k \neq c \end{cases}$$

Softmax Gradient: The Punchline

Claim: For the correct class c:

$$\frac{\partial \mathcal{J}}{\partial z_k} = p_k - y_k = \begin{cases} p_k - 1 & \text{if } k = c \\ p_k & \text{if } k \neq c \end{cases}$$

Visual: True class is $c = 2$

$$\underbrace{\begin{bmatrix} 0.2 \\ 0.7 \\ 0.1 \end{bmatrix}}_{\mathbf{p}} - \underbrace{\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}}_{\mathbf{y}} = \underbrace{\begin{bmatrix} +0.2 \\ -0.3 \\ +0.1 \end{bmatrix}}_{\nabla_{\mathbf{z}} \mathcal{J}}$$

Softmax Gradient: The Punchline

Claim: For the correct class c:

$$\frac{\partial \mathcal{J}}{\partial z_k} = p_k - y_k = \begin{cases} p_k - 1 & \text{if } k = c \\ p_k & \text{if } k \neq c \end{cases}$$

Visual: True class is $c = 2$

$$\underbrace{\begin{bmatrix} 0.2 \\ 0.7 \\ 0.1 \end{bmatrix}}_{\mathbf{p}} - \underbrace{\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}}_{\mathbf{y}} = \underbrace{\begin{bmatrix} +0.2 \\ -0.3 \\ +0.1 \end{bmatrix}}_{\nabla_{\mathbf{z}} \mathcal{J}}$$

Gradient descent pushes:

- z_2 **up** (negative gradient) — correct class
- z_1, z_3 **down** (positive gradient) — wrong classes

Softmax Gradient: Weight Update

Full gradient w.r.t. weights:

$$\nabla_{\mathbf{w}_k} \mathcal{J} = (p_k - y_k) \cdot \mathbf{x}$$

Softmax Gradient: Weight Update

Full gradient w.r.t. weights:

$$\nabla_{\mathbf{w}_k} \mathcal{J} = (p_k - y_k) \cdot \mathbf{x}$$

Same form as logistic regression!

- Correct class ($k = c$): gradient $= (p_c - 1)\mathbf{x}$, negative, increases $\mathbf{w}_c \cdot \mathbf{x}$
- Wrong classes ($k \neq c$): gradient $= p_k \mathbf{x}$, positive, decreases $\mathbf{w}_k \cdot \mathbf{x}$

Softmax Gradient: Weight Update

Full gradient w.r.t. weights:

$$\nabla_{\mathbf{w}_k} \mathcal{J} = (p_k - y_k) \cdot \mathbf{x}$$

Same form as logistic regression!

- Correct class ($k = c$): gradient $= (p_c - 1)\mathbf{x}$, negative, increases $\mathbf{w}_c \cdot \mathbf{x}$
- Wrong classes ($k \neq c$): gradient $= p_k \mathbf{x}$, positive, decreases $\mathbf{w}_k \cdot \mathbf{x}$

Magnitude: proportional to $|p_k - y_k|$
Confidently wrong $\Rightarrow$ big gradient $\Rightarrow$ big correction

Deriving the Softmax Gradient (Optional)

Want: $\frac{\partial \mathcal{J}}{\partial z_k}$ where $\mathcal{J} = -\log p_c$ and $p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}$

Deriving the Softmax Gradient (Optional)

Want: $\frac{\partial \mathcal{J}}{\partial z_k}$ where $\mathcal{J} = -\log p_c$ and $p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}$

Case 1: $k = c$ (the correct class)

$$\begin{aligned}\frac{\partial \mathcal{J}}{\partial z_c} &= -\frac{\partial}{\partial z_c}\left(z_c - \log \sum_j e^{z_j}\right) \\ &= -1 + \frac{e^{z_c}}{\sum_j e^{z_j}} = -1 + p_c = p_c - 1\end{aligned}$$

Deriving the Softmax Gradient (Optional)

Want: $\frac{\partial \mathcal{J}}{\partial z_k}$ where $\mathcal{J} = -\log p_c$ and $p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}$

Case 1: $k = c$ (the correct class)

$$\begin{aligned}\frac{\partial \mathcal{J}}{\partial z_c} &= -\frac{\partial}{\partial z_c}\left(z_c - \log \sum_j e^{z_j}\right) \\ &= -1 + \frac{e^{z_c}}{\sum_j e^{z_j}} = -1 + p_c = p_c - 1\end{aligned}$$

Case 2: $k \neq c$ (wrong class)

$$\begin{aligned}\frac{\partial \mathcal{J}}{\partial z_k} &= -\frac{\partial}{\partial z_k}\left(z_c - \log \sum_j e^{z_j}\right) \\ &= 0 + \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k\end{aligned}$$

Perceptron vs. Logistic Regression vs. Softmax

	Perceptron	**Logistic Reg.**	**Softmax**
Classes	2	2	$K \geq 2$
Output	± 1	$P(y=1) \in (0,1)$	$P(y=k)$ for all k
Loss	None	Binary CE	Multi-class CE
Gradient	$y\mathbf{x}$ (on error)	$(p-y)\mathbf{x}$	$(p_k - y_k)\mathbf{x}$
Convergence	Separable only	Always	Always

Perceptron vs. Logistic Regression vs. Softmax

	Perceptron	**Logistic Reg.**	**Softmax**
Classes	2	2	$K \geq 2$
Output	± 1	$P(y=1) \in (0,1)$	$P(y=k)$ for all k
Loss	None	Binary CE	Multi-class CE
Gradient	$y\mathbf{x}$ (on error)	$(p-y)\mathbf{x}$	$(p_k - y_k)\mathbf{x}$
Convergence	Separable only	Always	Always

Key insight: All three share the same gradient form: (prediction $-$ truth) $\times$ input

Perceptron vs. Logistic Regression vs. Softmax

	Perceptron	**Logistic Reg.**	**Softmax**
Classes	2	2	$K \geq 2$
Output	± 1	$P(y=1) \in (0,1)$	$P(y=k)$ for all k
Loss	None	Binary CE	Multi-class CE
Gradient	$y\mathbf{x}$ (on error)	$(p-y)\mathbf{x}$	$(p_k - y_k)\mathbf{x}$
Convergence	Separable only	Always	Always

Key insight: All three share the same gradient form: (prediction $-$ truth) $\times$ input

Softmax is the natural multi-class extension. When $K = 2$, softmax = logistic regression.

Practice Problem 1: Perceptron Trace

Sentiment classification with bag-of-words features:

Features: [count("good"), count("bad"), count("movie"), 1]

Review	good	bad	movie	bias	y
"good movie"	1	0	1	1	$+1$
"bad bad movie"	0	2	1	1	-1
"good good good"	3	0	0	1	$+1$

Initialize $\mathbf{w} = (0, 0, 0, 0)$

Tasks:

1. Trace through one epoch — which examples cause updates?
2. What are the final weights? Do they make intuitive sense?
3. Does the algorithm converge? How many epochs?

Practice Problem 2: Gradient Computation

Spam detection with features: [has_"free", has_"meeting", email_length, bias]

Training example: "Free money now!!!"
$\mathbf{x} = (1, 0, 3, 1), \quad y = 1$ (spam)

Current weights: $\mathbf{w} = (0.5, -0.3, 0.1, -0.2)$

Tasks:

1. Compute $z = \mathbf{w} \cdot \mathbf{x}$
2. Compute $p = \sigma(z)$ (use $\sigma(0.6) \approx 0.646$)
3. Compute the cross-entropy loss for this example
4. Compute the gradient $\nabla_{\mathbf{w}} \mathcal{J}$
5. Apply one SGD update with $\eta = 0.5$. Which weights increase? Does this make sense?

Practice Problem 3: Cross-Entropy Intuition

Consider two models predicting on the same test set:

Example	True y	Model A p	Model B p
1	1	0.9	0.7
2	0	0.2	0.3
3	1	0.6	0.8

Tasks:

1. Both models have 100% accuracy. Why isn't accuracy enough?
2. Compute cross-entropy loss for each model
3. Which model is "better" and why?

Practice Problem 4: Softmax Computation

3-class classification with scores:

$$z_1 = 2, \quad z_2 = 1, \quad z_3 = 0$$

True label: class 1

Tasks:

1. Compute p_1, p_2, p_3 using softmax
   (Hint: $e^0 = 1$, $e^1 \approx 2.72$, $e^2 \approx 7.39$)
2. Compute the cross-entropy loss $\mathcal{J} = -\log p_1$
3. Compute $\frac{\partial \mathcal{J}}{\partial z_k}$ for each k
4. Which scores should increase? Which should decrease?

Practice Problem 5: Sigmoid-Softmax Equivalence

Show that: For $K = 2$ classes, softmax reduces to sigmoid.

Setup: Two classes with scores z_1 and z_2

$$P(y = 1) = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

Why this matters: Logistic regression is just softmax with $K = 2$. One framework for all classification!

Discussion Questions

1. Why is cross-entropy loss preferred over squared error $(p - y)^2$ for classification?

2. Softmax with very large scores (e.g., $z_k = 1000$) causes numerical issues. How would you fix this? (Hint: what can you subtract from all z_k without changing the output?)

3. Why do we need K weight vectors for softmax but only 1 for logistic regression, even though both can do 2-class classification?

Key Takeaways

Perceptron:

- Simple, elegant, historically important
- Guaranteed convergence only for separable data
- Update: rotate $\mathbf{w}$ toward/away from misclassified points

Key Takeaways

Perceptron:

- Simple, elegant, historically important
- Guaranteed convergence only for separable data
- Update: rotate $\mathbf{w}$ toward/away from misclassified points

Logistic Regression:

- Probabilistic interpretation via sigmoid
- Cross-entropy loss from MLE; gradient: $(p - y) \cdot \mathbf{x}$

Key Takeaways

Perceptron:

- Simple, elegant, historically important
- Guaranteed convergence only for separable data
- Update: rotate $\mathbf{w}$ toward/away from misclassified points

Logistic Regression:

- Probabilistic interpretation via sigmoid
- Cross-entropy loss from MLE; gradient: $(p - y) \cdot \mathbf{x}$

Softmax:

- Multi-class extension; same gradient form: $(p_k - y_k) \cdot \mathbf{x}$
- Reduces to logistic regression when $K = 2$

Key Takeaways

Perceptron:

- Simple, elegant, historically important
- Guaranteed convergence only for separable data
- Update: rotate $\mathbf{w}$ toward/away from misclassified points

Logistic Regression:

- Probabilistic interpretation via sigmoid
- Cross-entropy loss from MLE; gradient: $(p - y) \cdot \mathbf{x}$

Softmax:

- Multi-class extension; same gradient form: $(p_k - y_k) \cdot \mathbf{x}$
- Reduces to logistic regression when $K = 2$

All are linear classifiers — next up: what if we stack them?

Questions?

Email: timothyobiso@brandeis.edu

Office Hours:
Tuesday 1:00-2:00 pm
Wednesday 2:00-3:00 pm